quick and easy way to check that your ROS system is configured correctly. To start the talker run the following command:

```
source /opt/ros/eloquent/setup.bash
ros2 run demo_nodes_cpp talker
```

If everything is working correctly you should see something like the following:

```
[INFO] [talker]: Publishing: 'Hello World: 1'
[INFO] [talker]: Publishing: 'Hello World: 2'
[INFO] [talker]: Publishing: 'Hello World: 3'
....
```

Now, let's fire up the listener. We're going to use a Python listener in this example to make sure we installed Python correctly. First we will need a second terminal. We can open a new terminal tab by entering `CTRL-SHIFT-T` in our terminal. We can also create a wholly new terminal by pressing `CTRL-ALT-T`. Pick whatever works best for you. Now in your new terminal source your bash file and run the following command:

```
source /opt/ros/eloquent/setup.bash
ros2 run demo_nodes_py listener
```

If everything is working correctly you should see something like the following:

```
[INFO] [listener]: I heard: [Hello World: 264]
[INFO] [listener]: I heard: [Hello World: 265]
[INFO] [listener]: I heard: [Hello World: 266]
```

Now that we have tested our ROS installation we can stop these two programs. In ROS most programs run in infinite loops until the robot is shut down. To stop these programs we navigate to the terminal running the program and press the `Ctrl` and `C` keys simultaneously. We call this combo `CTRL-C` and you can use it to stop just about any program in a terminal. Use it to stop the talker and listener programs.

Found a bug? Edit this page on GitHub.

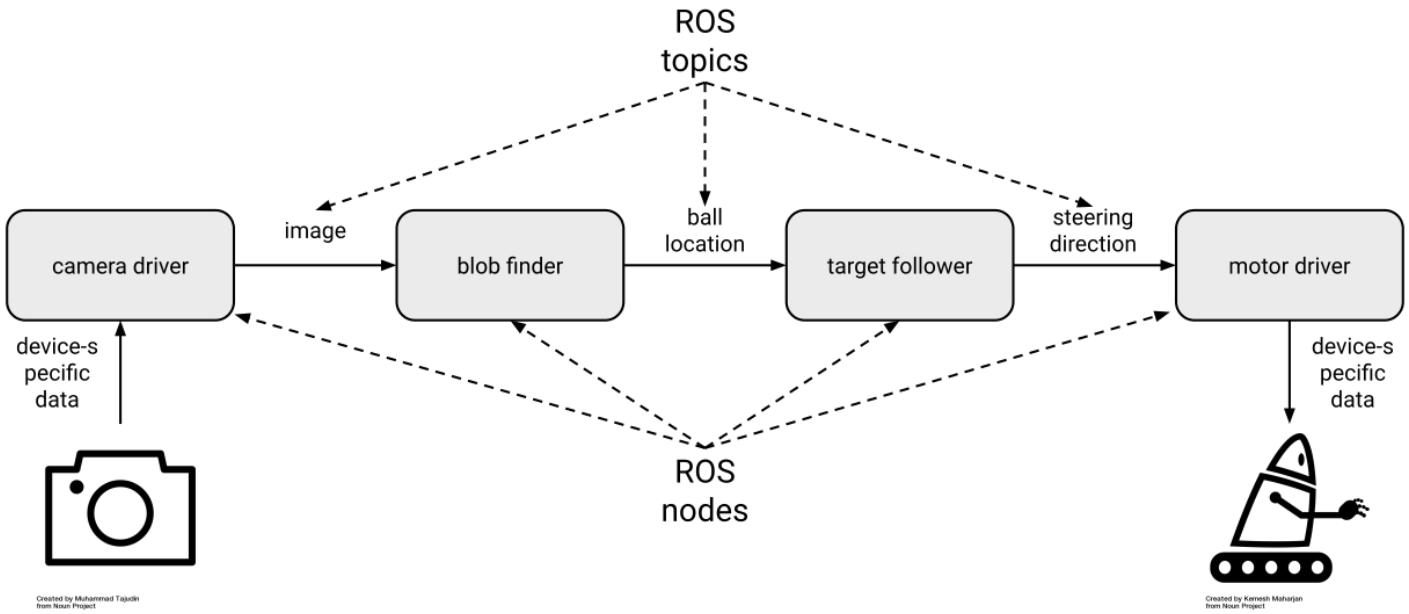
ROS Concepts and Design Patterns

As we said, learning about ROS is similar to learning about an automobile. In fact, a car is a lot like a robot (and sometimes it really is a robot; see the large and active self-driving vehicle industry). A modern automobile comprises many parts that are connected to each other. The steering wheel is connected to the front axle, the brake pedal is connected to the brake calipers, the oxygen sensor is connected to the fuel injectors, and so on. From this perspective, a car is a *distributed system*: each part plays a well-defined role, communicating (whether electrically or mechanically) as needed with other parts, and the result of that symphony-like coordination is a working vehicle.

A key philosophical tenet of ROS is that robotics software should also be designed and developed as a distributed system. We aim to separate the functions of a complex system into individual parts that interact with each other to produce the desired behavior of that system. In ROS we call those parts *nodes* and we call the interactions between them *topics* (and sometimes *services*, but we will get to that).

The ROS Communication Graph

Imagine we are building a wheeled robot that chases a red ball. This robot needs a camera with which to see the ball, a vision system to process the camera images to figure out where the ball is, a control system to decide what direction to move, and some motors to move the wheels to allow it to move toward the ball. Using ROS we might construct the system like so:



This design separates the software into four ROS *nodes*: two device drivers and two algorithms. Those nodes communicate with each other as shown, via three ROS *topics*. We call this structure a *ROS communication graph*: the nodes are the graph vertices and the topics are the graph edges. You can tell a lot about a ROS system by examining its communication graph.

The camera driver node is responsible for handling the details of interacting with the physical camera, which might happen through a custom USB protocol, through a vendor-provided library, or in some other way. Whatever those details, they are encapsulated inside the camera driver node, which presents a standard *topic* interface to the rest of the system. As a result, the blob finder node does not need to know anything about the camera; it simply receives image data in a standard format that is used for all cameras in ROS. The output of the blob finder is the detected location of the red ball, also in a standard format. Then the target follower node can read in the ball location and produce the steering direction needed to move toward the ball, again in a standard format. Finally, the motor driver node's responsibility is to convert the desired steering direction into the specific instructions necessary to command the robot's wheel motors accordingly.

Publish-Subscribe Messaging: Topics and Types

With the example of the ball-chasing robot in mind, we can add some terminology to describe what is happening as the system operates. First, the ROS communication graph is based on a well-known pattern called *publish-subscribe messaging*, or simply *pub-sub*. In a pub-sub system, as the name implies, data are sent as *messages* from *publishers* to *subscribers*. A publisher may have zero, one, or multiple subscribers listening to its published messages. Messages may be published at any time, making the system *asynchronous*.

In ROS, nodes publish and subscribe via topics, each of which has a name and a type. A publisher announces that it will be publishing data by *advertising* a topic. For example, the camera driver node may advertise a topic named `/image` with type `sensor_msgs/Image`. If the blob finder node subscribes to a topic with the same name and type, then the two nodes find each other and establish a connection over which image messages can get from the camera driver to the blob finder (the nodes find each other and establish those connections in a process called *discovery*, which will be covered in detail later in this book). Each message that flows across the `/image` topic will be of type `sensor_msgs/Image`.

A single node can be (and often is) both a publisher and a subscriber. In our example, the blob finder subscribes to image messages and publishes ball location messages. Similarly the target follower subscribes to ball location messages and publishes steering direction messages.

A topic's type is very important. In fact, taken together, the ROS types are among the most valuable aspects of the entire platform. First, a type tells you the syntax: which fields, of which

types, does the message contain? Second, it tells you the semantics: what do those fields mean and how should they be interpreted? For example, a thermometer and a pressure sensor might produce what appear to be the same data: a floating-point value. But in ROS, a well-designed thermometer driver node would publish one clearly defined type (say, `sensor_msgs/Temperature`), while a pressure sensor driver node would publish another (say, `sensor_msgs/FluidPressure`).

We always advise the use of semantically meaningful message types. For example, ROS provides simple message types like `std_msgs/Float64`, which contains a single 64-bit floating-point field called `data`. But you should only use that sort of generic type for rapid prototyping and experimenting. When you build a real system, even if something like `std_msgs/Float64` could get the job done on syntax, you should instead find or define a message that also matches the semantics of your application.

Why Publish-Subscribe?

Given that it comes with additional complexity (nodes, topics, types, etc.), it is reasonable to ask why ROS follows the pub-sub pattern. After more than a decade of building and deploying ROS-based robot systems, we can identify several key benefits:

- **Substitution**: If we decide to upgrade the robot's camera, we need only modify or replace the camera driver node. The rest of the system never knew the details of the camera anyway. Similarly, if we find a better blob finder node, then we can just swap it in for the old one and nothing else changes.
- **Reuse**: A well-designed blob finder node can be used today on this robot to chase the red ball, then reused tomorrow on a different robot to chase an orange cat, and so on. Each new use of a node should require only configuration (no code) changes.
- **Collaboration**: By cleanly separating concerns between nodes, we let our blob finder expert do their work independently of the target follower expert, with neither of them bothering the device driver expert. It is often the case that a robot application requires the combined expertise of many people, and it would be difficult to overstate the importance of ensuring that they can each contribute confidently and efficiently.
- **Introspection**: Because the nodes are explicitly communicating with each other via topics, we can listen in. So when the robot fails to chase the red ball, and we think that the problem is in the blob finder, we can use developer tools to visualize, log, and play back that node's inputs and outputs. The ability to introspect a running system in this way is instrumental to being able to debug it.
- **Fault tolerance**: Say that the target follower node crashes because of a bug. If it is running in its own process, then that crash will not bring down the rest of the system, and we can get things working again by just restarting the target follower. In general with ROS

we have the choice to run nodes in separate processes, which allows for such fault tolerance, or run them together in a single process, which can provide higher performance (and of course we can mix and match the two approaches).

- **Language independence**: It can happen that our blob finder expert writes their computer vision code in C++, while our target follower expert is dedicated to Python. We can accommodate those preferences easily by just running those nodes in separate processes. In ROS, it is perfectly reasonable, and in fact quite common, to mix and match the use of languages in this way.

Beyond Topics: Services, Actions, and Parameters

Most ROS data flows over topics, which we introduced in the previous sections. Topics are best for streaming data, which includes a lot of the common use cases in robotics. For example, going back to our ball-chasing robot, most cameras will naturally produces a stream of images at some rate, say, 30Hz. So it makes sense for the camera driver to publish the ROS messages containing those images just as soon as they're received. Then the blob finder will be receiving image messages at 30Hz, so it might as well publish its ball location messages at the same rate, and so on, through the target follower to the motor driver. We might say that such a systems is *clocked from the camera*: the data rate of the primary sensor, the camera in this case, drives the rate of computation of the system, with each node reacting in to receipt of messages published via topics by other nodes. This approach is fairly common and is appropriate for systems like our ball-chasing robot. There is no reason to do any work until you have a new camera image, and once you have one you want to process it as quickly as possible and then command an appropriate steering direction in response.

(We are making various simplifying assumptions, including that there is sufficient computational capacity to run all the nodes fast enough to keep up with the camera's data rate; that we do not have a way to predict where the ball is going in between camera frames; and that the motors can be commanded at the same rate the camera produces images.)

Services

So topics get the job done for the basic ball-chasing robot. But now say that we want to add the ability to periodically capture an ultra-high-resolution image. The camera can do it, but it requires interrupting the usual image stream that we rely on for the application, so we only want it to happen on demand. This kind of interaction is a poor fit for the publish-subscribe pattern of a topic. Fortunately, ROS also offers a request-reply pattern in a second concept: *services*.

A ROS service is a form of remote procedure call (RPC), a common concept in distributed systems. Calling a ROS service is similar to calling a normal function in a library via a code API. But because the call may be dispatched to another process or even another machine on the network, there is more to it than just copying pointers around. Specifically, a ROS service is implemented using a pair of ROS messages: a *request* and a *reply*. The node calling the service populates the request message and sends it to the node implementing the service, where the request is processed, resulting in a reply message that is sent back.

We might implement the new high-res snapshot capability like so:

- **Define a new service type.** Because services are less widely used than topics, there are relatively few "standard" service types predefined. In our case, the new service's request message might include the desired resolution of the snapshot. The request message could be a standard `sensor_msgs/Image`.
- **Implement the service.** In the camera driver, we would advertise the newly defined service so that when a request is received, the usual image-handling is interrupted temporarily to allow the device interaction necessary to grab one high-res snapshot, which is then packed into a reply message and sent back to the node that called the service.
- **Call the service.** In the target follower node, we might add a timer so that every 5 minutes, it calls the new service. The target follower would receive the high-res snapshot in response to each call, and could then, say, add it to a photo gallery on disk.

In general, if you have a need for infrequent, on-demand interactions among nodes, ROS services are a good choice.

Actions

Sometimes, when building robot control systems, there is a need for an interaction that looks like request-reply, but that can require a lot of time between the request and the reply. Imagine that we want to wrap up our ball-chasing control system into a black box that can be invoked as part of a larger system that makes the robot play football. In this case, the higher level football controller will periodically want to say, "please chase the red ball until you have it right in front of you." Once the ball is in front of the robot, the football controller wants to stop the ball-chasing controller and invoke the ball-catching controller.

We *could* achieve this kind of interaction with a ROS service. We could define a chase-ball service and implement it in the target follower. Then the football controller could call that service when it wants the ball chased. But ball-chasing may take quite some time to complete, and it may fail to complete. Unfortunately, after calling the chase-ball service, the football controller is stuck waiting for the reply, similar to the situation in which you call a long-running

function in code. The football controller does not know how well (or poorly) the chase is going, and it cannot stop the chase.

For such goal-oriented, time-extended tasks, ROS offers a third concept that is similar to services but more capable: *actions*. A ROS action is defined by three ROS messages: a goal, a result, and feedback. The goal, sent once by the node calling the action to initiate the interaction, indicates what the action is trying to achieve; for ball-chasing it might be the minimum required distance to the ball. The result, sent once by the node implementing the action after the action is complete, indicates what happened; for ball-chasing it might be final distance to the ball after the chase. The feedback, sent periodically by the node implementing the action until it is complete, updates the caller on how things are going; for ball-chasing it might be the current distance to the ball during the chase. In addition, actions are cancelable, so the football controller can decide to give up and move onto another tactic if the chase is taking too long or if the feedback messages are showing that there is little chance of success.

In general, if you want to support on-demand, long-running behaviors, ROS actions are a good choice.

Parameters

Any nontrivial system requires configuration, and ROS is no exception. When we start our robot's motor driver node, how do we tell it to connect to the motors via `/dev/ttyUSB1`? We do not want to hard-code that information into the node, because on the next robot it might be `/dev/ttyUSB0` instead. ROS addresses such configuration needs via a fourth concept: *parameters*. A ROS parameter is what you might expect: a named, typed, place to store a piece of data. For example, the motor driver node may define a parameter called `serial_port` with type string. When it starts up, the node would use the value of that parameter to know which device to open to get to the motor system.

ROS parameters can be set in a few ways:

- **Defaults.** A ROS node that uses a parameter must embed in its code some default value for that parameter. In the case that nothing else in the system sets the parameter value explicitly, the node needs some value to work with.
- **Command-line.** There is standard syntax for setting parameter values on the command-line when launching a node. Values set in this manner override defaults in the code.
- **Launch files.** When launching nodes via the `launch` tool instead of manually via the command-line, you can set parameter values in the launch file. Values set in this manner override defaults in the code.
- **Service calls.** ROS parameters are dynamically reconfigurable via a standard ROS service interface, allowing them to be changed on the fly, if the node hosting the parameters allows it. Values set in this manner override whatever previous values were set.

For most nodes, parameter management is relatively simple: define a handful of parameters, each with a reasonable default; retrieve the parameters' values at startup, which accounts for changes made via command-line or launch file, then begin execution and disallow future changes. This pattern makes sense for the motor driver, which needs to know which `/dev/ttyUSB` device file to open at startup, and does not support changing that setting later. But there are cases that require more sophisticated handling. For example, the blob finder node may expose as parameters a variety of thresholds or other settings that configure how it identifies the red ball in images. These kinds of settings can be changed on the fly, which the target follower might want to do, based on how well the chase is going. In this case the blob finder needs to be sure to use the latest values for its parameters, knowing that they may have been changed by another node.

In general, when you want to store stable, but possibly changeable, configuration information in a node, ROS parameters are a good choice.

Asynchrony in Code: Callbacks

Throughout ROS, you will see a common pattern in the code, which is the use of *callback functions*, or simply *callbacks*. For example, when subscribing to a topic, you supply a callback, which is a function that will be invoked each time your node receives a message on that topic. Similarly, when you advertise a service, you supply a callback that is invoked when the service is called. The same goes for actions (for handling of goals, results, and feedback) and parameters (for handling of setting new values).

Programming with callbacks is not familiar to everyone. It differs from the standard sequential presentation of programming, in which you write a `main()` function that does A, then B, then C, and so on. By contrast, in ROS (and in most systems that focus on data-processing and/or control), we follow an event-based pattern. In this pattern, we do A whenever X happens, B whenever Y happens, and so on.

A common structure for a ROS node is the following:

- **Get parameter values.** Retrieve the node's configuration, considering defaults and what may have been passed in from outside.
- **Configure.** Do whatever is necessary to configure the node, like establish connections to hardware devices.
- **Set up ROS interfaces.** Advertise topics, services, and/or actions, and subscribe to services. Each of these steps supplies a callback function that is registered by ROS for later invocation.
- **Spin.** Now that everything is configured and ready to go, hand control over to ROS. As messages flow in and out, ROS will invoke the callbacks you registered.

Following this structure, a `main()` function in a ROS node is often very short: initialize and configure everything, then call a spin function to let ROS take over. When you are trying to understand what is happening in a ROS node, look in the callbacks; that is where the real work is happening.

Found a bug? Edit this page on GitHub.

The ROS Command Line Interface

The ROS command line interface, or CLI for short, is a set of programs for starting, inspecting, controlling, and monitoring a ROS robot. The best way to think of the CLI is a collection of small and simple programs that allow you perform basic tasks in ROS. Drawing from our car analogy, the CLI can be thought of as the subsystems of a vehicle: the breaks, the transmission, the window wipers, all of the smaller parts that are composed together to build the larger vehicle. What we'll show you in this section is how to turn on the car, put it in gear, turn on the radio, and perhaps check your oil to perform routine maintenance. The ROS 2 CLI draws heavily from the Unix/Linux philosophy of small programs that can be composed together. If you are familiar with the command line interface found in Unix and Linux, or to a lesser extent in MacOS or Windows, you'll feel right at home.

The ROS command line tools draw heavily from the design patterns mentioned in the previous section, and directly interface with the APIs we will address in the next section. The CLI interface is at its core just a set of simple tools built from the ROS 2 API; this API is simply an implementation of the high-level patterns we discussed in the previous section. If your goal is to simply interface with a particular piece of software written using ROS, the CLI interface is the way you will go about starting, stopping, and controlling the underlying ROS software. For more advanced users these tools will allow you to study a ROS system by exploring the underlying software processes in the system.

There are only two things you need to memorize from this section. The first command simply tells your computer that you are using ROS, and what version of ROS you want to use. Let's take a look at the magic command:

```
source /opt/ros/eloquent/setup.bash
```

If everything is working correctly, this command should simply return. Nothing happens that you can see, but underneath the hood you've just told this particular shell that you are using ROS 2 Eloquent Elusor, and where all the ROS programs and files live. You should plan on doing this every time you want to use ROS. The most common mistake new users have is not running this command. If you're not sure if you ran the command in a shell, that's okay. The command is idempotent; running it twice in a row won't break anything. You can run it a million times in a row and it won't make any difference.

The other command you need to commit to memory is `ros2`. Almost everything in the ROS 2 CLI starts with `ros2`. Go ahead and try it in the same shell where you just sourced the setup file. If everything is working correctly you should see the following: